The Journal of Systems and Software 113 (2016) 173–187

Contents lists available at ScienceDirect

The Journal of Systems and Software

journal homepage: www.elsevier.com/locate/jss

Mobile device power models for energy efficient dynamic offloading at runtime

Farhan Azmat Ali [a,*], Pieter Simoens [a,b], Tim Verbelen [a], Piet Demeester [a], Bart Dhoedt [a]

[a] Ghent University – iMinds, Department of Information Technology, Gaston Crommenlaan 8, Ghent 9050, Belgium
[b] Ghent University, Department of IT & C, Valentin Vaerwyckweg 1, Ghent 9000, Belgium

ARTICLE INFO

Article history:
Received 3 February 2015
Revised 20 November 2015
Accepted 21 November 2015
Available online 30 November 2015

Keywords:
Power model
Energy consumption
Energy-aware dynamic offloading

ABSTRACT

Spectacular advances in hardware and software technologies have resulted in powerful mobile devices, equipped with advanced processing, storage and network capabilities. Therefore, using resource-intensive applications has become a commodity in many contexts. However, the rapid evolution in hardware and software capabilities has not been paralleled by a similar advance in battery technology. A potential avenue to cope with the device energy resource limitation is to offload computational tasks to cloud infrastructure in the network. In order to offload tasks in an energy-aware manner, we present a detailed model of mobile device energy consumption, addressing the main power consuming subsystems, including CPU, display unit, wireless network interface and memory. Applying this model allows to estimate the power consumed by the application when executed locally, remotely or hybridly (i.e. partly on the device and partly in the cloud infrastructure). Offloading parts of the application can subsequently be decided at runtime based on these energy consumption estimates, also taking into account the power consumed by the device-to-cloud communication over the wireless network. The dynamic offloading has been validated with computational and communication intensive applications. Results show that 18–55% energy gains on the mobile device can be achieved, depending on different conditions.

1. Introduction

Smartphones and tablets have rapidly become the preferred devices for consuming communication, business and entertainment applications. Users heavily rely on their handheld devices to perform a wide range of daily activities, including e-mail correspondence, web browsing, internet banking and watching streamed videos. Nowadays, these devices are typically equipped with powerful multicore processors, giga-bytes of memory, and are connected to the internet via fast communication technologies such as Long-Term Evolution (LTE) or Wi-Fi. Unlike spectacular advances in hardware technologies for these devices, battery technologies innovate at a much slower pace. In the period from 2000 to 2012, battery capacities of handheld devices have doubled, while processing capabilities increased by a factor of up to 12 (Atluri et al., 2012) . Notwithstanding these advances in hardware technologies, mobile device still suffer from limitations in computing and storage capabilities when executing complex, resource-intensive applications such as rich multimedia applications, advanced data analysis applications and gaming, etc.

A solution to the resource and battery constraints consists of offloading computationally intensive tasks to high performance machines hosted in the network (Vallina-Rodriguez and Crowcroft, 2013; Kumar and Lu, 2010). Offloading computation can result in significant speedups of computation and conserve battery power, and therefore offloading to a remote server effectively broadens the range of applications that can be accessed from the mobile device (Vallina-Rodriguez and Crowcroft, 2013).

The energy consumption of the mobile device CPU can be significantly reduced by offloading major processing tasks to a remote server using available communication channels. However, the need for data exchange between the mobile device and the remote server results in additional energy consumption. Consequently, in terms of energy consumption, offloading to the remote server is only beneficial if the energy for sending the request (including data involved) and receiving the response is lower than the energy needed for executing the task locally on the mobile device (Kumar and Lu, 2010). Another reason for offloading could be the potential reduction in response time, as the computation resources available at the remote server are typically an order of magnitude larger than the local device computational capacity (Ma et al., 2012). Here also care must be taken that the delay incurred by the network communication is smaller than the gain resulting from faster execution at the server side.

* Corresponding author. Tel.: +32 9 33 14938.
E-mail address: farhan.azmatali@intec.ugent.be (F.A. Ali).

http://dx.doi.org/10.1016/j.jss.2015.11.042
0164-1212/

Table 1
Comparison of previous work related to power modeling.

	CPU		Display unit	Memory	Network
	Cores	Auto-scaling			Wi-Fi
CloneCloud (Chun et al., 2011)	On–Off model		x		On–Off model
MAUI (Cuervo et al., 2010)	Single-core tested				Simplified[a]
Power Tutor (Zhang et al., 2010)	Single-core	2 freq. supported	x		Simplified[a]
AppScope (Yoon et al., 2012)	Single-core	x	x		Simplified[a]
PowerProf (Kjrgaard and Blunck, 2012)	Single-core	Not mentioned			Simplified[a]
Online Estimation (Kim et al., 2012)	Dual-core	x	x		Simplified[a]
Into the Wild (Shye et al., 2009)	Single-core	2 freq. supported	x		Simplified[a]
Our work	Multi-core support	x	x	x	Detailed[b]

[a] Wi-Fi simplified model does not differentiate between Send and Receive states.
[b] Wi-Fi detailed model takes into account Send, Receive, Tail and Idle states.

To decide which application components should be offloaded, it is essential to estimate the power consumption of different deployment configurations, including local execution of the application. To this end, a model for the energy consumed by different hardware subsystems is needed, more specifically for the CPU, the wireless interface and the memory. As the power consumed by the display is not affected by an offloading decision, modeling the display energy consumption is less crucial. However, for the sake of completeness, also a model for this subsystem is reported on in this paper.

The contribution of this paper is two-fold. Firstly, we present a comprehensive power consumption model for all subsystems of a mobile device including CPU, Wi-Fi, memory access (RAM) and display unit. The proposed model for the CPU power consumption takes into account auto-scaling and fine-grained utilization. The Wi-Fi model clearly differentiates between different states of the network interface card. The memory access model considers power consumption due to read/write memory operations by applications. Secondly, we explore dynamic offloading of computational intensive parts of applications based on these power models. Our energy aware offloading architecture AIOLOS (Verbelen et al., 2012) applies the proposed models to measure the energy consumption of application components at runtime and decides whether the component should be offloaded to a server or not. In summary, we present a comprehensive energy model integrated in a framework encompassing all system aspects relevant for computational offloading, and evaluate this model on practical use cases.

The remainder of this paper is structured as follows. In Section 2, we explore the related work, both on power models for mobile devices and computational offloading architectures. In Section 3, we elaborate on energy models for the different hardware subsystems. In Section 4, we detail our measurement approach, present the resulting power models and validate these using different scenarios. Section 5 provides a brief introduction on the AIOLOS offloading platform and Section 6 validates the concept of energy aware dynamic offloading on three applications.

2. Related work

2.1. Power models for mobile devices

Related work on modeling power consumption can be categorized along several dimensions (Kjrgaard and Blunck, 2012). A first dimension concerns the method used to collect the measurements for designing the model: measurements can be collected using external equipment or using an internal battery interface. A second dimension is the type of information from which the model is built. A power model can be designed from (1) measurements of system utilization, e.g. processor statistics available from OS level (2) power consumptions measurements per system call made to OS or (3) power consumption measurements per API call made in a specific programming language.

A number of research efforts have been conducted on the derivation of power models for different hardware building blocks of a mobile device, in order to assist application developers to estimate energy requirements of mobile application. PowerTutor (Zhang et al., 2010) uses information extracted from the voltage drop to determine the battery discharging rate to estimate the power consumption. These models are constructed and validated for single-core mobile processors. As our measurements indicate, applying the same models for modern high performance multi-core mobile devices would be rather inaccurate. Also, the Wi-Fi model does not differentiate between the power consumed in the sending and the receiving states, yielding less accurate results.

The solution described in Shye et al. (2009) uses a background logger that periodically monitors the resource utilization by stressing different hardware components of the mobile device. A linear regression model is designed by combining current measurements with an external device, with, OS reported battery operating voltage and logger statistics. However, the model is designed for a single core CPU and lower CPU frequencies are ignored for designing the power model which is rather imprecise for the offloading use case.

PowerProf (Kjrgaard and Blunck, 2012) utilizes a smart battery to measure the energy consumption at runtime via a special API and this information is used to measure the energy usage of application API calls. Using genetic algorithms (Kantardzic, 2003), an energy usage profile is determined for each method call. This approach is only applicable when a smart battery implementing the special API is available and it assumes that each time a method is called the same amount of energy will be used, which is often an oversimplification.

AppScope (Yoon et al., 2012) is an Android-based energy metering system that uses power models and usage statistics for each hardware component. Linear regression power models are designed using DevScope (Jung et al., 2012)and hardware resource usage monitoring is acquired by loading additional modules into the Linux kernel. To estimate the usage statistics of each application, traces and inter-process communication are analyzed, resulting in considerable overhead. Also this tool does not handle multi-core processors.

The approach discussed in Kim et al. (2012) proposes advanced online power estimation techniques for multi-core smart phones using an external power meter. Power models are derived based on the relationship between subsystem utilization and their power consumption. However, the CPU power model is designed and validated for dual-core CPU's only, and the Wi-Fi model does not differentiate between Send and Receive States of the wireless interface.

Existing power models, as summarized in Table 1, lack one or more features of modern mobile hardware: they are designed for single core CPUs, ignore energy proportionality of CPU frequency scaling or do not differentiate between the different power states of network interface cards. Moreover, special hardware (smart battery) and software APIs are required to measure the energy consumption of application API calls.

Table 2
Comparison of previous work related to computational offloading.

	Energy aware	Energy savings	Call back	Partitioning	Static/dynamic
Cuckoo (Kemp et al., 2012)	Always offload	40% (theoretical)		Android service only	Static
CloneCloud (Chun et al., 2011)	x (Constrained execution time)	20%		Code partition	Static (profiling based)
MAUI (Cuervo et al., 2010)	x	20%-video game 45%-chess game		Method offloading	Dynamic
ThinkAir (Kosta et al., 2012)	x			Method offloading	Dynamic
MISCO (Dou et al., 2010)	N.A.			Map/reduce	Dynamic
COSMOS (Shi et al., 2014)	N.A.			Method offloading	Dynamic
Honeybee (Fernando et al., 2013)	x (Constrained execution time)			Parallel independent task	Dynamic
Serendipity (Shi et al., 2012)	x			Parallel independent task	Dynamic
Dynamic Execution (Gao et al., 2014)	x	60%-chess walk 45%-Firefox		Method offloading	Dynamic
AIOLOS (Verbelen et al., 2012)	x	43%-test app. 55%-chess game	x	OSGi service offloading	Dynamic

Our power model copes with all these limitations. It is validated for multi-core CPUs and all supported frequencies are considered. Furthermore, Wi-Fi power model clearly differentiates between different states of the wireless interface card. No special hardware or software support is required, and there is no extra overhead involved in analyzing different traces. Our model also considers the energy consumption due to memory access (RAM) of the application.

2.2. Computation offloading

The concept of transferring computationally intensive tasks to remote servers to improve mobile application's performance and reduce local energy cost has attracted considerable research interest from the mobile computing community. Different offloading architectures, as summarized in Table 2, tend to offload application's method calls with a minimum of programmer efforts required to change application's logic. Offloading parts of an application can be decided at runtime depending upon energy requirements of local and remote method execution. Mobile device profiling is mostly performed off-line.

MAUI (Cuervo et al., 2010) supports fine-grained code offloading while minimizing the changes required to the application. It uses extensive off-line profiling to decide to offload method calls on the Microsoft.NET runtime platform. The goal is to minimize the energy usage, MAUI profiles the mobile device and builds a linear model to estimate the energy consumption of a piece of code based on number of CPU cycles this code requires to execute.

CloneCloud (Chun et al., 2011) is an Android-oriented platform that enables access to cloud resources to augment the mobile device's computational power without requiring the programmer to explicitly partition the application. The platform enables applications to offload part of their execution from the mobile device to device clones in the cloud implemented through application-layer virtual machines such as Dalvik VM, JAVA VM and .NET. However, the partitioning mechanism in CloneCloud is off-line.

Both MAUI and CloneCloud energy estimation models are imprecise because these models do not take into account a number of important factors influencing a sound offloading decision, as mentioned in Jiao et al. (2013). These factors include: (1) the energy consumption of I/O operations; (2) auto-scaling CPU clock speeds; (depending on load conditions, the CPU frequency is adapted, having an impact on the execution time of a given code fragment).

Cuckoo (Kemp et al., 2012) offers a complete framework for computational offloading for Android and simple programming model which is integrated into the Eclipse build system. It bundles both local and remote execution code in a single package and allows existing applications to offload service components. However, this approach does not support callbacks from the offloaded code, always prefers remote execution and does not offer run-time dynamic offloading based on energy consumption.

ThinkAir (Kosta et al., 2012) is method level framework for offloading computation exploiting the concept of smartphone virtualization. It profiles hardware state information to construct an energy estimation model similar to PowerTutor (Zhang et al., 2010), tracks parameters mainly method execution time and determines the current network state. ThinkAir focuses on different static offloading profiles to make the offloading decision and takes into account execution time, energy consumption and cloud infrastructure utilization.

MISCO (Dou et al., 2010) is an implementation of MapReduce-paradigm for the mobile context which splits a large computational task into smaller independent tasks that can run on other mobile devices. The application developer focuses on the implementation of the map and reduce functions, while the framework itself handles all the scheduling, data flow, failure and parallel execution of applications. The framework does not consider energy estimation when deciding where to execute a map and reduce task.

COSMOS (Shi et al., 2014) identifies compute-intensive tasks of an application in the same way as MAUI (Cuervo et al., 2010) and CloneCloud (Chun et al., 2011). The offloading decision is made based on computational speedup and communication delay. Based on this, COSMOS estimates how long an offloaded task will need in order to complete processing and return results (response time). The cloud resource information is shared among mobile clients and each client randomly chooses a cloud server to offload the task. However, COSMOS does not provide energy-aware task offloading.

Honeybee (Fernando et al., 2013) is a crowd-computing framework built on Android which enables mobile devices to share work, utilize local resources and human collaboration in the mobile context. Bluetooth is a communication medium between delegator and worker. The framework is designed for the tasks that can be broken down into several independent jobs. The framework decides to offload tasks based on the speedup factor. However, the framework does not provide energy aware offloading and does not support callback support.

Serendipity (Shi et al., 2012) enables a mobile device to use other mobile computational resource to speedup computational tasks and to conserve energy of the task initiating device. Each network node constructs its device profile (based on execution speed and energy consumption model using PowerTutor (Zhang et al., 2010)), and

shares and maintains profiles of other encountered nodes. The execution profile of a program is determined using an offline method similar to MAUI (Cuervo et al., 2010) and CloneCloud (Chun et al., 2011). Device and job profiles are used to estimate the jobs execution time and energy consumption on each node in the network, based on which the task allocation is done.

An order-k semi-Markov framework is presented in Gao et al. (2014), which formulates the stochastic transitions among application methods, and provides quantitative guidelines on offloading decisions through probabilistic estimations of the offloading effectiveness. Profiling is based on each method execution time, and power consumptions is known using a hardware power meter. The application programmer is responsible to specify whether a method is remoteable or not. To offload method call, the CloneCloud (Chun et al., 2011) approach is used and information collection about execution time and energy consumption as suggested in MAUI (Cuervo et al., 2010).

AIOLOS (Verbelen et al., 2012) is a cyber foraging framework that uses approaches both at component and method level. The offloading decision is made for each method offered by the components and the framework autonomically selects the method to offload. The framework optimizes execution time of each method based on the size of method call's arguments and the network characteristics. To achieve this, estimates are derived from a sliding window of observed duration of previous remote and local calls, using a random sample consensus algorithm. Previous versions of the framework optimized execution time; in this paper, we extend the framework to optimize energy consumption. We construct power models for smart phones taking into account multi-core processor architectures as well as the different states of the wireless interface card (especially differentiating between sending and receiving states, and also taking into account tail state). Experiments are performed with external power meter to have fine-grained analysis of power consumption due to different hardware components. We have chosen this approach as most of the smartphones just offer information "how much battery is left" API that is very coarse-grained and often unreliable. The power models can be constructed with minimal overhead, as they basically extract information from the */proc* file system and do not require sophisticated trace analysis. Furthermore, constructing these power models does not require any modifications at the kernel level. The derived power models are subsequently validated for different offloading scenarios where application's task are offloaded by taking into account energy consumption of that task locally and remotely.

3. Mobile phone energy model

The hardware components of interest include the CPU, the display unit, the RAM, sensors and the network interface. To determine the energy consumption, the activity state of each component must be varied between two extreme states whilst keeping the energy consumption of other components constant. We focused on hardware components affected by the offloading decision, e.g. CPU, memory usage (RAM) and Wi-Fi, and have excluded components that cannot be offloaded (e.g. GPS, bluetooth, audio, camera, storage device). As mentioned in the introduction, also the power consumed by the display unit is modeled.

3.1. CPU

The CPU power consumption depends upon its operating frequency and utilization. CPU's typically use dynamic frequency scaling techniques adapting the CPU clock frequency on the fly according to the load (lowering the clock speed to conserve energy, increasing the clock speed to enhance the performance).

In general, the power consumption by the multi-core CPU can be modeled as

$$Power_{cpu} = \sum_{j=0}^{n-1} \left(\beta_j^{freq} \times \mathrm{U}_j\right) + \beta_{base}^{freq} \tag{1}$$

where n is the total number of active CPU cores, U_j is the utilization for core j, and β_j^{freq} and β_{base}^{freq} are frequency dependent coefficients. The parameter β_{base}^{freq} denotes the minimal CPU power consumption in idle state.

However, in practice it is difficult to exactly model the CPU power consumption on a per core basis as for some devices it is not possible to disable cores (keeping one core active and disable other cores) to measure the power consumption of a single core only. Other CPU cores can be activated anytime (depending on CPU scheduler) and remain in idle state influencing the overall power consumption of CPU. Therefore we have considered total CPU utilization as shown in equation

$$Power_{cpu} = \beta^{freq} \times \mathrm{U} + \beta_{base}^{freq} \tag{2}$$

where U is the aggregated CPU utilization

$$U = \sum_{j=0}^{n-1} U_j \tag{3}$$

U_j the CPU utilization per CPU core and n is the total number of active CPU cores. As will be shown in the experiments in Section 6, using this aggregated CPU utilization parameter (U) results in a good approximation of power actually consumed by the CPU.

3.2. Display unit

The display unit plays an important role in the human-to-machine interaction, and its power consumption heavily depends on the brightness level, the actual displayed content (e.g. a white pixel takes more energy than a black pixel), the display size and resolution. Variations in power consumption mainly stem from changing the display's brightness level and from displaying dynamic content. We approximate the power consumed by the device by

$$Power_{lcd} = \left(\frac{\alpha_{black} + \alpha_{white}}{2} \times B_{level}\right) + \left(\frac{\alpha_{base.black} + \alpha_{base.white}}{2}\right) \tag{4}$$

where α_{black}, α_{white}, $\alpha_{base.black}$ and $\alpha_{base.white}$ are power coefficients, expressing the power consumption as a function of the brightness level for a black and white pixel respectively. The $\alpha_{base.black}$ and $\alpha_{base.white}$ coefficients exhibiting the minimal power consumed by the display unit when display contents (illuminated pixels) are complete black and white respectively with minimal brightness level (where displayed contents are still visible).To estimate power consumption accurately, one should of course know the detailed screen content to be displayed in advance, which is not realistic. Therefore, we take an average of the power consumed by a white and a black pixel as a per pixel power consumption approximation. As will be shown in the experimental results, this approach allows for realistic power consumption estimates. The parameter B_{level} represents the brightness level of the display unit, and varies from 0 to 255.

3.3. Wi-Fi

Wi-Fi is a major source of energy consumption in smartphones. The consumed power is mainly dependent on four parameters: (1) the number of packets sent and (2) received per second, (3) the channel rate and (4) data rate (Zhang et al., 2010).

We assume that channel and the data rate variables are constants and stable during the experiments, and we focus only on the number

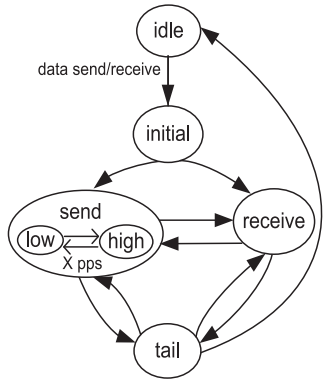


Fig. 1. Wireless network interface card (WNIC) state model: the wireless card changes to different states depending on the data transmitted and/or received.

of packets exchanged per second to derive a Wi-Fi power model. Note that this model is limited to conditions where the signal strength of the wireless connection is sufficiently high, and the network connection itself is stable. Offloading in harsh network conditions is not considered, as it is difficult to predict performance gains and to make energy trade-offs in unstable and uncertain network environments. In principle, the energy model could be extended to worse network conditions, taking the observed signal strength as a model parameter (requiring to build a Wi-Fi model for different signal strength values). As indicated in Ou et al. (2015), both network throughput and energy consumed by the WNIC are severely affected when the signal strength falls below a certain threshold (typically −79 dBm). As we assume a relatively good wireless network connection in order to consider offloading, this extension of the Wi-Fi model is not addressed in this paper and mentioned as the future work.

The wireless network interface card (WNIC) has four power states which are detailed as below. Fig. 1 exhibits the state transition diagram and captures the estimation of power behavior of the WNIC.

- *Idle State:* The WNIC does not perform any operation in this state and remains in low power state.
- *Initial State:* The WNIC enters into Initial State if it was in Idle State and wants to exchange data. The WNIC powers up the necessary hardware components to send or receive data.
- *Send State:* This state is entered when the WNIC actually sends the data. The duration of this state depends upon the number of packets sent per second. The Send State has two substates in terms of power consumption: low and high power state (Zhang et al., 2010). The card enters into a substate depending upon the number of packets sent per second. Transition from low to high power state happens when the number of packets sent per second exceeds a given threshold. Likewise, if the number of packets sent drop below this threshold, the WNIC returns to low power state again. The card can directly switch to Receive State if it wants to receive data.
- *Receive State:* This state is entered when the WNIC receives data. The number of packets per second influences the duration of this state. The card can directly switch to Send State if it wants to send data.
- *Tail State:* After exchange of data, the WNIC triggers a tail-timer which is started each time a packet is sent/received. In this state, the WNIC is still ready to exchange data and immediately switches back to either the Send or Receive state. Tail State is observed for WNIC in Pathak et al. (2011) and also for LTE (Huang et al., 2012; Bornholt et al., 2012).

The full Wi-Fi energy model is a combination of the energy consumption model for each of the different states, which is in turn affected by the number of packets exchanged (N) per second and by the time spent in that state, yielding

$$Energy_{wifi} = \begin{cases} Energy_{wifi\text{-}init} + Energy_{wifi\text{-}S} \\ +Energy_{wifi\text{-}R} + Energy_{wifi\text{-}tail} \\ +Energy_{wifi\text{-}idle} \end{cases} \quad (5)$$

$Energy_{wifi\text{-}init}$, $Energy_{wifi\text{-}S}$, $Energy_{wifi\text{-}R}$, $Energy_{wifi\text{-}tail}$ and $Energy_{wifi\text{-}idle}$ represent energy consumed in Initial, Send, Receive, Tail and Idle state, respectively.

3.4. Device memory [RAM]

Memory access contributes up to 20% of the total power consumption of mobile device (Sklavos and Touliou, 2007). The power consumed by accessing the memory heavily depends on the type of application (i.e. the memory access pattern).

We approximate the power consumption of the RAM by

$$Power_{memory} = \gamma_S \times U + \gamma_m \quad (6)$$

where γ_S and γ_m are power coefficients, while U represents the aggregated CPU utilization and can be determined from Eq. (3).

4. Energy characterization of smartphones

In this section, we present our methodology to extract the model parameters, together with results on two smart phones, i.e. Samsung Galaxy S2 and Galaxy S3, equipped with a dual-core and a quad-core processor, respectively. Detailed specifications of both devices are presented in Table 3.

Table 3
Hardware specification of mobile devices.

Hardware component	Samsung Galaxy S2	Samsung Galaxy S3
CPU	1.2 GHz dual-core ARM Cortex-A9	1.4 GHz quad-core ARM Cortex-A9
GPU	ARM Mali-400	ARM Mali-400/MP4
LCD	Super AMOLED	Super AMOLED
Wi-Fi	802.11 bgn	802.11 abgn
GPS	A-GPS and stand-alone GPS	A-GPS and stand-alone GPS
Cellular	2G GSM: 850/900/1800/1900 HSDPA: 850/900/1900/2100	2G GSM: 850/900/1800/1900 HSDPA: 850/900/1900/2100
Bluetooth	3.0 + HS	4.0 with A2DP and EDR
Audio	Built-in microphone and speaker	Built-in microphone and speaker
Camera	8 MP	8 MP
Battery	Li-ion 1650 mAh	Li-ion 2100 mAh

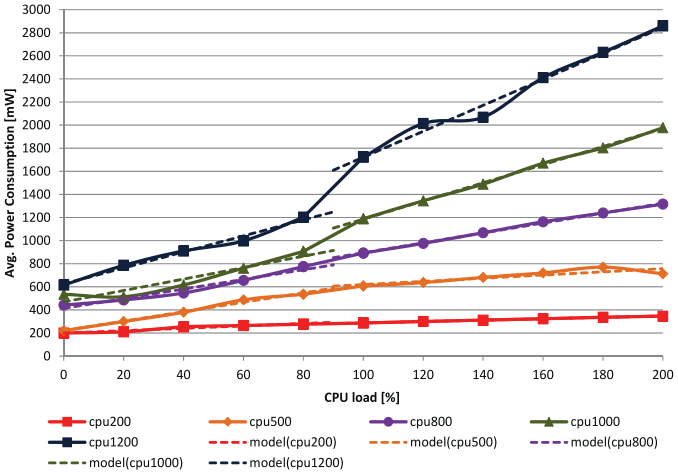
4.1. Measurement approach

To accurately measure the power consumed by the mobile device, the device power monitor from Monsoon Inc. is used (Monsoon, 2013). In these experiments, an external power source replaces the battery of the mobile device, and the voltage and current delivered to the mobile device are monitored at a sampling frequency of 5 kHz. Post-processing of the logs produced by the power meter allows detailed, time-resolved estimations of the actual power consumed by the device.

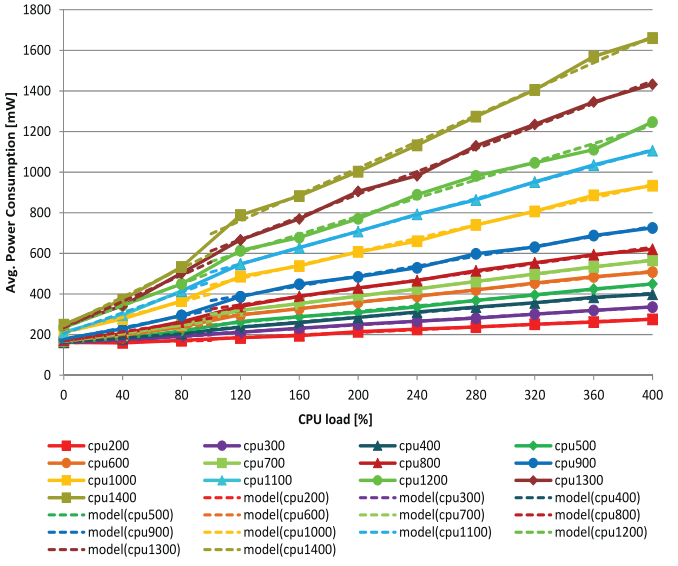
To derive model parameters for the subsystems discussed in the previous section, scripts were executed sequentially, each script focusing on varying the utilization of one subsystem, whilst keeping the other subsystems in a stable state.

4.2. Measurement results

In this subsection, we will characterize two mobile devices Samsung Galaxy S2 and Galaxy S3. All measurement parameters for these

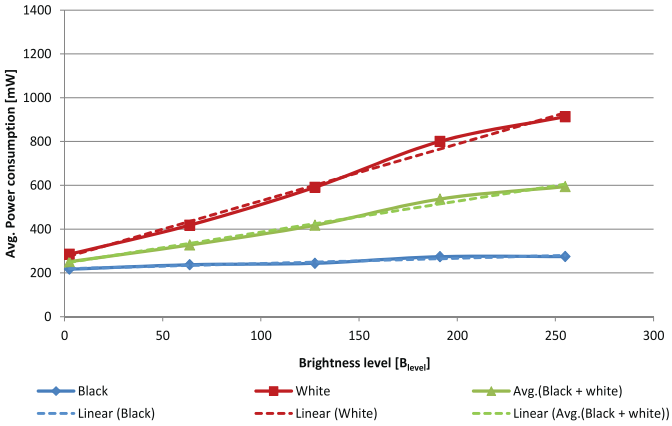


(a) CPU power model for dual-core mobile device (S2)

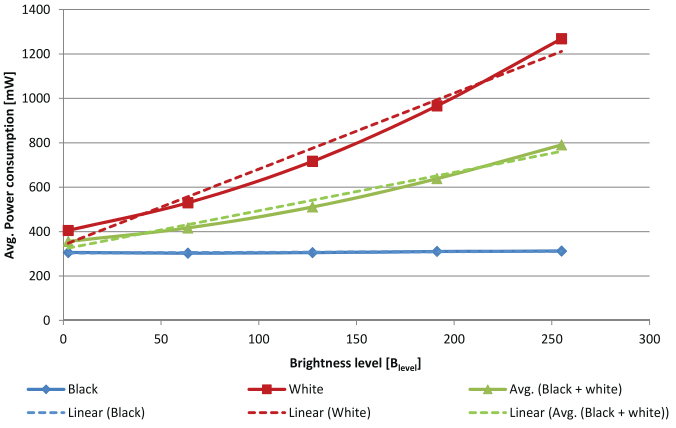


(b) CPU power model for quad-core mobile device (S3)

Fig. 2. CPU power modeling for dual-core (S2) and quad-core (S3): power consumption of different frequencies with variation in CPU utilization.



(a) Display unit power model for Galaxy S2



(b) Display unit power model for Galaxy S3

Fig. 3. Display unit power modeling of Galaxy S2 and S3: power consumption of display unit with variation in brightness level for different screen colors. (For interpretation of the references to color in this figure legend, the reader is referred to the web version of this article.)

devices are presented in Tables 4 and 5, respectively. As we have assumed linear models for power consumption, all coefficients were determined using a standard linear fit approach.

4.2.1. CPU

As discussed in the previous section, two parameters dominate the power consumed by the CPU: the clock frequency of the CPU and its utilization. In Android, the frequency scaling rules are defined by *CPUfreq Governors* (2013). The governor defines the power characteristics of the system CPU, which in turn affects CPU performance. Each governor has its own unique behavior, purpose, and suitability in terms of workload. These power governors are *performance, powersave, ondemand, userspace* and *conservative*. In order to isolate the influence of the clock frequency, the frequency is fixed using *userspace* frequency scaling governor.

The CPU utilization is gradually varied between idle (0%) and maximum (200% for dual-core and 400% for quad-core) in steps (20% for dual-core and 40% for quad-core) using the *userspace* power governor. All CPU clock frequencies supported by the device are evaluated, ranging to 1200 MHz for the S2 and 1400 MHz for the S3 device respectively. All other hardware components (including cellular networking, Wi-Fi networking, display unit, ...) are disabled.

To generate artificial load on the CPU, a dedicated Android background activity was designed, that changes the load generated at fixed time intervals. This activity is designed multi-threaded to gradually increase the CPU load and eventually activate multiple cores of the CPU.

The actually generated CPU load is monitored by inspecting the */proc* file system, and results are logged, to be correlated with power consumption logs. The total CPU load, the load on all cores, is considered to model CPU power as many applications can have multi-threaded activities.

Experimental results shown in Fig. 2a and 2b indeed indicate a close-to-linear relationship between the CPU utilization and power consumption for both devices. For the S2-device (Fig. 2a), we observe that the second core is activated when CPU load is between 80% and 100%, irrespective of the clock frequency.

Results for the quad-core device are exhibited in Fig. 2b. We notice that the CPU of Galaxy S3 is more power efficient than Galaxy S2 at all measured frequencies. For example, the power consumption by the S2 at 200% utilization and 1200 MHz frequency is around 2800 mW while for these utilization and clock frequency, the S3 only consumes 800 mW. Even if we double the CPU utilization (400%)for the S3, the S3 CPU consumes only 1200 mW which is

Table 4
Power model coefficients: dual-core Galaxy S2.

Freq. [MHz]	CPU Utilization U [%]	Coefficient β^{freq} [mW]	Coefficient β^{freq}_{base} [mW]	State	Wi-Fi Power	Coefficient	Value [mW]
200	0–90	1.0389	199.50	Initial		$I_{S2\text{-}w}$	1.9043
	91–200	0.6001	227.59			$I_{S2\text{-}b}$	69.89
500	0–90	4.0828	221.32		p_{low}	$S_{S2\text{-}w1}$	12.916
	91–200	1.376	481.91	Send		$S_{S2\text{-}b1}$	536.61
800	0–90	4.178	413.93		p_{high}	$S_{S2\text{-}w2}$	0.80
	91–200	4.2873	466.23			S_{S2-b2}	767.89
1000	0–90	4.9521	469.07		p_{low}	$R_{S2\text{-}w1}$	1.6045
	91–200	7.8525	401.39	Receive		$R_{S2\text{-}b1}$	560.11
1200	0–90	6.9305	625.25		p_{high}	$R_{S2\text{-}w2}$	0.1711
	91–200	11.248	597.29			$R_{S2\text{-}b2}$	598.11
	Pixel	Coefficient	Value [mW]	Tail		T_{S2}	187
	Black	α_{black}	0.2406	Idle		ID_{S2}	5
		$\alpha_{base.black}$	218.56		Time	Coefficient	Value [ms]
Display unit	White	α_{white}	2.5916	Send		$St_{S2\text{-}wt}$	0.4632
		$\alpha_{base.white}$	269.90			$St_{S2\text{-}bt2}$	1.6007
	Avg.	$\frac{\alpha_{black}+\alpha_{white}}{2}$	1.4161		t_1	$Rt_{S2\text{-}wt1}$	0.5067
	b+w	$\frac{\alpha_{base.black}+\alpha_{base.white}}{2}$	244.23	Receive		$Rt_{S2\text{-}bt1}$	0.0509
					t_2	$Rt_{S2\text{-}wt2}$	0.0485
						$Rt_{S2\text{-}bt2}$	11.287
				Initial		*S2.init.time*	10
				Tail		*S2.tail.timer*	202

Table 5
Power model: quad-core Galaxy S3.

Freq. [MHz]	CPU Utilization U [Range]	Coefficient β^{freq} [mW]	Coefficient β^{freq}_{base} [mW]	State	Power	Wi-Fi Coefficient	Value [mW]
200	0–100	0.1207	159.24	Initial		$I_{S3\text{-}w}$	0.8613
	101–400	0.3271	145.62			$I_{S3\text{-}b}$	98.612
300	0–100	0.3955	159.04		p_{low}	$S_{S3\text{-}w1}$	−0.6285
	101–400	0.4404	159.81			$S_{S3\text{-}b1}$	661.13
400	0–100	0.5289	161.84	Send	p_{high}	$S_{S3\text{-}w2}$	0.4049
	101–400	0.5953	166.13			$S_{S3\text{-}b2}$	686.93
500	0–100	0.6964	159.2	Receive	p	$R_{S3\text{-}w}$	0.0638
	101–400	0.6757	178.95			$R_{S3\text{-}b}$	336.67
600	0–100	0.865	166.25	Tail		T_{S3}	195
	101–400	0.7667	205.62	Idle		ID_{S3}	20
700	0–100	0.9884	165.89		Time	Coefficient	Value [ms]
	101–400	0.8906	211.66				
800	0–100	1.1352	167.92	Send		$St_{S3\text{-}wt}$	0.617
	101–400	1.0153	224.48			$St_{S3\text{-}bt}$	0.1245
900	0–100	1.4387	176.49		t_1	$Rt_{S3\text{-}wt1}$	0.1158
	101–400	1.2151	245.66	Receive		$Rt_{S3\text{-}bt1}$	6.490
1000	0–100	1.9244	208.03		t_2	$Rt_{S3\text{-}wt2}$	0.0211
	101–400	1.6513	278.05			$Rt_{S3\text{-}bt2}$	15.628
1100	0–100	1.4714	277.22	Initial		*S3.init.time*	10
	101–400	2.0103	306.54	Tail		*S3.tail.timer*	202
1200	0–100	2.6897	235.16				
	101–400	2.236	335.88			Display unit	
1300	0–100	3.2275	233.9		Pixel	Coefficient	Value [mW]
	101–400	2.7922	332.78				
1400	0–100	3.5447	242.78		Black	α_{black}	0.0316
	101–400	3.2407	371.99			$\alpha_{base.black}$	303.38
	Memory				White	α_{white}	3.4219
	Coefficient	Value [mW]				$\alpha_{base.white}$	339.37
	γ_{S3}	6.1465			Avg.	$\frac{\alpha_{black}+\alpha_{white}}{2}$	1.7267
	$\gamma_{M\text{-}S3}$	256			b+w	$\frac{\alpha_{base.black}+\alpha_{base.white}}{2}$	321.37

less than half of the power consumed by the S2. For the quad-core device, we observe that the CPU activates additional cores when its utilization is between 80% and 120% (irrespective of the clock frequency).

4.2.2. Display unit

The power consumed by the display unit is influenced by the brightness level set by the user, which can be easily monitored, as well as by the actual content of the display (i.e. the status of the pixels). By taking into account the actual display content when estimating the power consumption at runtime, an avyeraging approach was taken: for each brightness level, the average between two extremes was measured, i.e. a completely white screen on the one hand and a completely black screen on the other hand, as explained in Section 3.2.

Results for both extremes, as well as the average, as a function of brightness are shown in Fig. 3a and 3b for the S2 and S3 devices, respectively. From these results, we observe that the display unit of S3

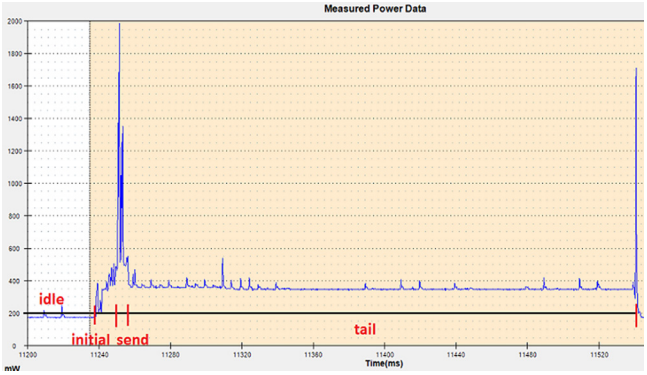


Fig. 4. Power meter trace exhibiting different states of Wi-Fi send energy model for mobile device (S3), where number of sent packets are $N = 4$.

consumes considerably more power than the S2 with factor 1.2 (i.e. $Power_{lcd\text{-}S3} = Power_{lcd\text{-}S2} * 1.2$). Both S2 and S3 have super AMOLED screens, and the difference in power consumptions is mainly due to the higher resolution and larger screen size of the S3 device.

4.2.3. Wi-Fi

To model the Wi-Fi power consumption, experiments were conducted, varying the number of packets per time unit. Packets are exchanged between the local server and mobile device at different packet rates, e.g. 4, 8, 12, 16, 20, 25, 50, 75 and 100 packets per second (N), with constant packet size (1450 bytes) using UDP as transport protocol. Different packet rates were chosen to determine the threshold value between low and high power states as explained in Section 3.3. Exchange of network traffic is observed by inspecting the */proc* file system of the Android kernel at 1 Hz.

The total power consumed by the WNIC consists of power consumed in each state as described in Section 3.3. These states are Initial State, Send State, Receive State, Tail State and Idle State.

The WNIC switches from Idle State to Initial State before actual data transmission to power up necessary hardware components. After sending all data packets, the WNIC briefly remains in the Send State before switching to the Tailing State which is also a power consuming state. Subsequently, the WNIC moves back to Idle State again if no data is exchanged during the Tail State.

The energy consumption of the mobile device in Initial State is determined by following equation

$$Energy_{Sx.wifi\text{-}init} = \begin{cases} (I_{Sx\text{-}w} \times N + I_{Sx\text{-}b}) \times Sx.init.time & \text{if } N > 0 \\ 0 & \text{if } N = 0 \end{cases} \quad (7)$$

where $I_{Sx\text{-}w}$, $I_{Sx\text{-}b}$ and $Sx.init.time$ are the model coefficients, while N stands for the number of packets sent or received per second.

After the Initial State, the WNIC enters into either Send State or the Receive State. The energy consumption in the Send State is determined by the following equation:

$$Energy_{Sx.wifi\text{-}S} = \begin{cases} 0 & \text{if } N = 0 \\ (S_{Sx\text{-}w1} \times N + S_{Sx\text{-}b1}) \times Sx.send.time & \text{if } 0 < N \leq 20 \\ (S_{Sx\text{-}w2} \times N + S_{Sx\text{-}b2}) \times Sx.send.time & \text{if } 20 < N \end{cases} \quad (8)$$

where $S_{Sx\text{-}w1}$, $S_{Sx\text{-}b1}$, $S_{Sx\text{-}w2}$ and $S_{Sx\text{-}b2}$ are the model coefficients, while N is number of packets sent per second.

Energy consumption in the Send State depends upon number of packets sent which decides whether the WNIC will enter either in Send-low or Send-high power state. The WNIC will enter into low power state if number of packets sent per second is $N \leq 20$. Otherwise, the WNIC will enter in high power state. This threshold value is determined by experimentation and was also reported in Yoon et al. (2012). Fig. 4 represents a power consumption trace, and shows the different power consumption states.

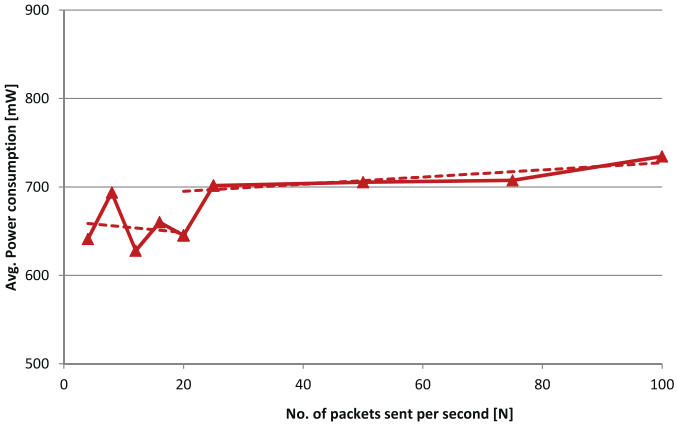


Fig. 5. Send power consumption comparison with threshold $N = 20$ for mobile device (S3).

Fig. 5 shows the send power consumption with threshold value. It is observed that there is a clear difference in power consumption above and below this threshold value.

$Sx.send.time$ is the time required to send the number of packets (N) per second. This parameter clearly depends upon N and can be determined from the following equation:

$$Sx.send.time = (St_{Sx\text{-}wt} \times N + St_{Sx\text{-}bt}) \qquad \text{if } N > 0 \quad (9)$$

where $St_{Sx\text{-}wt}$, and $St_{Sx\text{-}bt}$ are the model coefficients of equation.

Likewise, the energy consumption in Receive State can be determined by the following equation:

$$Energy_{Sx.wifi\text{-}R} = \begin{cases} 0 & \text{if } N = 0 \\ (R_{Sx\text{-}w} \times N + R_{Sx\text{-}b}) \times Sx.recv.time & \text{if } N > 0 \end{cases} \quad (10)$$

where $R_{Sx\text{-}w}$, and $R_{Sx\text{-}b}$ are the model coefficients. N stands for total number of packets received per second. $Sx.recv.time$ is the time required to receive the number of packets (N) per second and is determined by the following equation:

$$Sx.recv.time = \begin{cases} (Rt_{Sx\text{-}wt1} \times N + Rt_{Sx\text{-}bt1}) & \text{if } 0 < N \leq 20 \\ (Rt_{Sx\text{-}wt2} \times N + Rt_{Sx\text{-}bt2}) & \text{if } 20 < N \end{cases} \quad (11)$$

where $Rt_{S3\text{-}wt1}$, $Rt_{S3\text{-}bt1}$, $Rt_{S3\text{-}wt2}$ and $Rt_{S3\text{-}bt2}$ are coefficients.

As indicated in Ou et al. (2013), sending and receiving packets concurrently can reduce the power consumed by the WNIC, because the time the WNIC remains in the high power state is reduced. However, it is difficult to monitor whether packets are sent and received in a concurrently or sequentially and to include this effect in a reliable way into the power model. Therefore, we estimate the power consumed as if sending and receiving were sequentially, as a worst case estimate.

After data transmission, the WNIC switches to Tail State which is also a state of relatively high power consumption. The WNIC remains active in this state to switch back to the Send State or the Receive State until the tail-timer expires. The energy consumption in this state can be determined by the following equation:

$$Energy_{s3.wifi\text{-}tail} = T_{Sx} \times Sx.tail.time \quad (12)$$

where T_{Sx} is model coefficient. Average $S3.tail.time$ is 202 ms. However, it depends upon the fact that whether the WNIC has switched to the Send State or the Receive State during tail-timer or not. If the WNIC switches, then the tail-timer will be restarted. In order to measure energy consumption in this situation, we assume that the data is uniformly exchanged during the measured interval.

The energy consumption in Idle State can be determined by the following equation:

$$Energy_{Sx.wifi\text{-}idle} = ID_{Sx} \times Sx.idle.time \tag{13}$$

where ID_{S3} is the coefficient and $S3.idle.time$ is the time the WNIC remains in Idle State.

The total Wi-Fi energy consumption for the mobile device can be determined by combining Eqs. (7), (8), (10), (12) and (13)

$$T.Energy_{Sx.wifi} = \begin{cases} Energy_{Sx.wifi\text{-}init} + Energy_{Sx.wifi\text{-}S} \\ +Energy_{Sx.wifi\text{-}R} + Energy_{Sx.wifi\text{-}tail} \\ +Energy_{Sx.wifi\text{-}idle} \end{cases} \tag{14}$$

where $T.Energy_{S3.wifi}$ represents the total Wi-Fi energy consumed in different states.

For the S2 device, the Wi-Fi energy consumption for Initial State, Send State, Tail State and Idle State can be extracted from Eqs. (7), (8), (12) and (13), respectively, as the Wi-Fi power model for the S2 and S3 devices is similar. In this case, Galaxy S3 device coefficients will be replaced by the S2 coefficients as mentioned in Table 4. However, difference lies in the Receive State. For the S2 device, receive power changes at threshold value $N = 20$.

The S2 energy consumption in Receive State can be determined by following equation:

$$Energy_{S2.wifi-R} = \begin{cases} 0 & \text{if } N = 0 \\ (R_{S2\text{-}w1} \times N + R_{S2\text{-}b1}) & \text{if } 0 < N \leq 20 \\ \quad \times S2.recv.time & \\ (R_{S2\text{-}w2} \times N + R_{S2\text{-}b2}) & \text{if } 20 < N \\ \quad \times S2.recv.time & \end{cases} \tag{15}$$

$R_{S2\text{-}w1}$, $R_{S2\text{-}b1}$, $R_{S2\text{-}w2}$ and $R_{S2\text{-}b2}$ are the model coefficients, while N represents total packets received per second.

$S2.recv.time$ is time to receive the number of packets N per second and is determined by following equation:

$$S2.recv.time = \begin{cases} (Rt_{S2\text{-}wt1} \times N + Rt_{S2\text{-}bt1}) & \text{if } 0 < N \leq 20 \\ (Rt_{S2\text{-}wt2} \times N + Rt_{S2\text{-}bt2}) & \text{if } 20 < N \end{cases} \tag{16}$$

$Rt_{S2\text{-}wt1}$, $Rt_{S2\text{-}bt1}$, $Rt_{S2\text{-}wt2}$ and $Rt_{S2\text{-}bt2}$ are the coefficients.

4.2.4. Memory access [RAM]

The memory access behavior by an application is difficult to monitor, as typically no provisions are taken in the OS for this purpose. Therefore, we have to resort to a worst and best case analysis, characterizing the power consumed in a scenario with infrequent memory access and a scenario with heavy memory load. Experiments were conducted for Galaxy S3, varying the CPU utilization between 0% and 400%, while fixing the CPU clock frequency.

To generate artificial load on CPU with heavy memory read and write operations, two random number are generated. The generated numbers are stored in two separate arrays (memory write operation). The content of corresponding array elements is read (memory read operation), and added up yielding an other number stored in third array (memory write operation). The whole operation is looped depending upon the required CPU load.

Fig. 6 presents results for both scenarios, confirming that memory operations can indeed considerably contribute to the total power consumed by the device.

The power model for RAM access is described by

$$Power_{memory-S3} = \gamma_{S3} \times U + \gamma_{M\text{-}S3} \tag{17}$$

γ_{S3} and $\gamma_{M\text{-}S3}$ are the coefficient, while the aggregated CPU utilization (U) varies from 0% to 400% and can be determined from Eq. (3).

4.3. Model validation

To validate our proposed power models for CPU, Wi-Fi, RAM access and display unit etc., we choose a web browsing scenario as this scenario combines all subsystems described earlier. Experiments are performed using two client applications: a regular web browser and a client mimicking the web browsing functionality but not displaying the content on the screen.

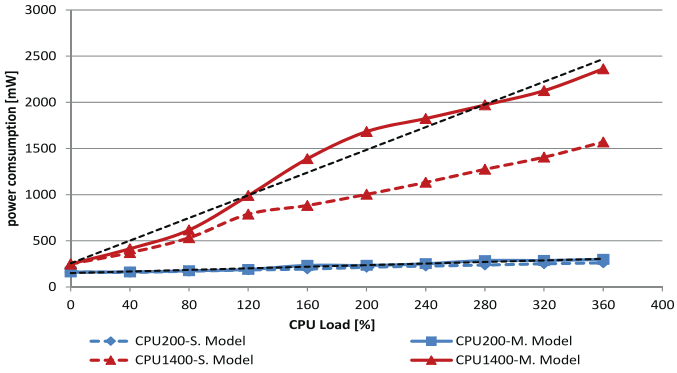


Fig. 6. Memory modeling [RAM]: power consumption due to frequent read and write operations to system's RAM for the Galaxy S3.

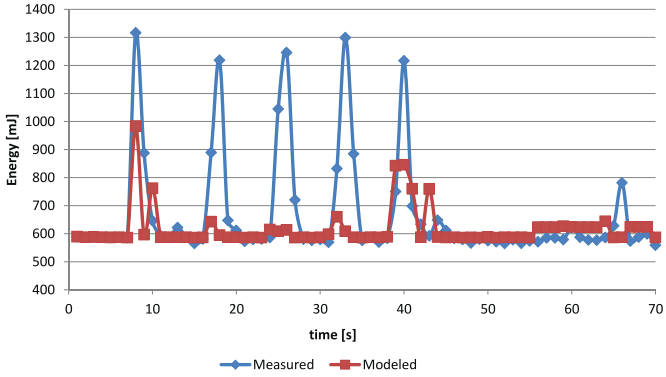
This was done to isolate the influence of the GPU power consumption. GPU power modeling is not the purpose of this paper, since the main objective of the work is to assess the impact of power considerations on offloading (GPU-intensive operations are related to display operations, and are hence not suitable for offloading). Both experiments are performed at minimum brightness level to eliminate the influence of the display power consumption.

The script used for validating the power model using web browsing fetches the content of a static web page. Results for browsing are presented in Fig. 7a. At the start of the browsing session, the user selects a bookmarked URL by touching screen and waits for web page to be completely downloaded and displayed on screen. This effect is represented by first peak in Fig. 7a. The graph shows that the modeled peak is less pronounced than the actually measured peak. This difference is due to the GPU activity necessary for rendering the display update on the screen.

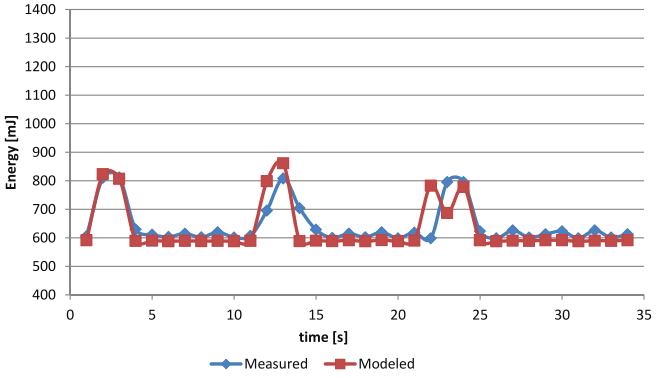
After this first displaying of the web page, the user interacts with the device by (i) scrolling towards the page end and (ii) scrolling to the left hand side of the web page, and (iii) zooming-in by pinching two fingers. The effects of these interactions are presented in Fig. 7a as second, third and fourth measured peaks respectively. The power consumption model is clearly unable to predict these peaks as these tasks are mainly performed by the GPU, which is not accounted for in the power model.

As a last step in the scenario, the user clicks on a hyper link to open another web page and this activity is exhibited by last peak in the Fig. 7a. The model is able to predict the last peak to some extent, mainly because of the data communication contribution, and rest of the energy is utilized by GPU to update the screen. The power consumed by the GPU obviously depends on type of application (and the interactions by the user) and accounts for up to 21% of total system power consumption (Kim and Chung, 2013). As we are concentrating on offloading, and GPU-related tasks are not suitable for offloading, it is less important to incorporate GPU power consumption in the trade-off between local and remote execution.

To confirm that the difference between model and the actually measured power consumption is due to GPU activity, a customized client was constructed, fetching the same content from the web, but not rendering this content on the display. Results are presented in Fig. 7b. In this case, power consumption peaks represent energy required to fetch the URL content from the web (i.e. CPU load) and downstream data traffic. In this case, we observe a close match between measured and modeled peaks. Average model error remains less than 8%.



(a) Regular browser: the user fetches page content from the web and navigates through the page by performing different activities, e.g. scrolling, zooming etc.



(b) Browsing client: screen updates are disabled to exclude the power consumed by the GPU

Fig. 7. Power model validation for web browsing scenario.

5. Dynamic offloading

As already mentioned in the introduction of the paper, the designed power models are used to estimate the energy consumed by the application when executed locally and remotely on the cloud. This energy consumption estimation provides input to decide which part of application should be offloaded at runtime. To support energy based dynamic offloading, we make use of the AIOLOS-framework, a Java-based offloading framework facilitating component offloading between Android smartphones and the remote cloud infrastructures (Verbelen et al., 2012).

This section details the offloading architecture of the AIOLOS cyber foraging framework to dynamically offload application's components to the cloud infrastructure.

5.1. Energy aware dynamic offloading

The approach of energy-aware dynamic offloading based on the power consumption models is presented in Fig. 8. These models estimate the energy consumed by different hardware components for different deployment configurations of an application. To this end, resource usage is continuously monitored on the mobile device, more specifically the status of the CPU, Wi-Fi and display unit. This information is provided to the power model, which estimates the energy consumption for a number of deployment options. In turn, the power model provides input to the deployment decision module, which takes the decision whether to execute the application locally or to offload some application components to a remote infrastructure. As

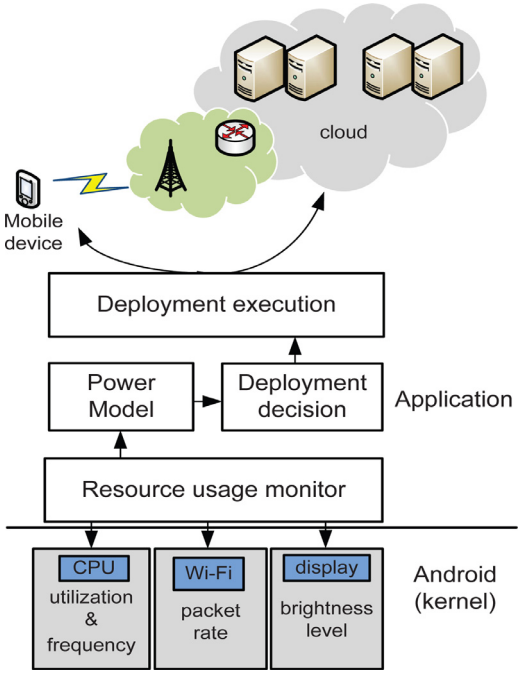


Fig. 8. Energy aware dynamic offloading architecture: power models estimate energy consumption based on hardware monitoring, providing input to deployment decision engine.

this decision is taken at runtime, platform support should be available for dynamic component offloading. In this paper, we choose Android and the Java programming language as underlying platform, in view of the availability of a suitable offloading platform AIOLOS (Verbelen et al., 2012). Nevertheless, the methodology presented and the resulting models can be readily applied to other mobile platforms and programming languages.

5.2. AIOLOS Architecture

AIOLOS (Verbelen et al., 2012) is a cyber foraging framework for Android, which is built on OSGi, a modular system and service platform in Java, enabling transparent monitoring and mobile code (The OSGi Alliance, 2009). In its original version, the framework optimizes execution time, and therefore monitors the time needed for local and remote execution for each method call. Based on this information, the framework chooses the most appropriate method execution location, and stores the resulting execution time in its knowledge base for future use. In this process, AIOLOS keeps track of the amount of data exchanged during the method call, in order to be able to estimate execution time in varying network conditions. The approach optimizes the application responsiveness, taking into account device capabilities and network connectivity, without putting an additional burden on the application developer. To facilitate remote method invocation, AIOLOS leverages the R-OSGi protocol (Rellermeyer et al., 2007).

The architecture of the AIOLOS framework is depicted in Fig. 9, and consists of the following components:

(1) The *Proxy Manager* is responsible for creating and managing proxies for each application bundle's services. By proxying each registered service, the middleware can on the one hand decide to forward a service call to a remote service implementation rather than to the local one, and on the other hand gather monitoring information of all service calls. In order to proxy services, the Proxy Manager uses Service Hooks (The OSGi Alliance, 2009). When an application bundle registers a service with the OSGi framework, the *Proxy Manager* gets

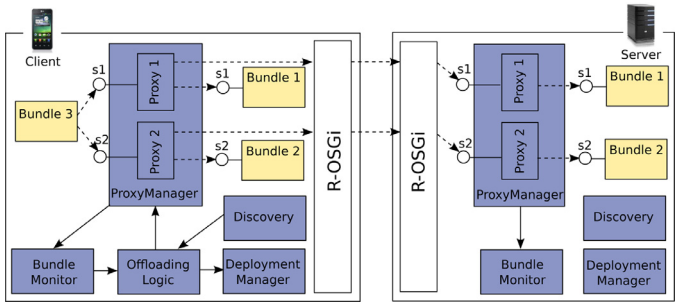


Fig. 9. The architecture of the AIOLOS cyber foraging framework.

a callback through the Listener Hook, and generates a proxy for this service which will also register the service with the OSGi service registry. When a bundle looks up the service in the OSGi service registry, the *Proxy Manager* can hook in using the Find Hook and returns the service registered by the proxy, hiding the original service implementation. In this way, all monitoring and remote execution happens transparently to all application bundles. This also allows to handle errors transparently, for example when the network connection is lost during remote execution, a time-out is detected by the proxy and the method can be executed locally.

(2) The *Bundle Monitor* gathers monitor information of all application bundles. When a service method is called, the *Bundle Monitor* gets notified by the involved proxy and captures the size of the input arguments, the size of the return value and the execution time of the method call. Power models are incorporated into *Bundle Monitor* which uses this information to estimate energy required to offload and execute a method remotely. The size of input arguments and return value provide input for energy estimation for data communication over the network, and execution time yields processing energy. Using this information, an energy based execution profile for each method is constructed, which is used to identify candidate methods for remote execution by the *Offloading Logic* bundle.

(3) The *Offloading Logic* implements the algorithm to decide whether to offload a method or not. Decision is made depending upon energy consumption of method call locally or remotely to the server. Method call would be offloaded to remote server if

$$E_{local} > E_{offload} \tag{18}$$

E_{local} is the energy consumed by the CPU of mobile device when a method is executed locally on the mobile device itself (no offloading).

$$E_{local} = E_{cpu} \tag{19}$$

$E_{offload}$ can be expressed as following:

$$E_{offload} = (E_{network} + E_{idle}) \tag{20}$$

$E_{network}$ represents network energy to offload method which depends upon argument size and return value of the method. E_{idle} is the energy consumption when mobile device remains idle while waiting for the processed data to be returned from the remote server.

The *Offloading Logic* will also instruct the *Deployment Manager* to copy bundles to the server when methods are identified that would benefit from offloading, in case the associated bundles are not available at the server yet.

The offloading decision should also consider QoE related metrics, e.g. delay, in addition to purely energy related figures. Therefore, the AIOLOS offloading framework (Verbelen et al., 2012) supports primitives allowing the delay to specify e.g. threshold on request-response delays. When AIOLOS (Verbelen et al., 2012) estimates that cloud-side execution of a method will not meet the delay bound specified, it will not consider offloading (irrespective of the possible energy gains).

(4) The *Deployment Manager* can migrate bundles to remote servers by sending the byte codes to the server and installing the bundle and its dependencies there on the OSGi runtime.

(5) The *Discovery* uses the Service Localization Protocol (SLP) to discover remote servers in the network that are suitable for offloading.

6. Power model validation with offloading use cases

To evaluate the effectiveness of dynamic offloading to conserve mobile device's power, three different types of applications are evaluated. A first application is a synthetic Test application used to offload computational task to remote server by varying data length exchanged between mobile device and the remote server. The second application is a computational intensive application where the application's major task is to execute a complex algorithm. Honza's Chess, 2013 application is selected which is an existing open source chess game developed for Android. The third application is data communication intensive where the application needs to upload significant amounts of data to a server to process and return back. The Photo Editor application is chosen which enables the user to perform several operations on pictures. Both Honza's Chess and the Photo Editor applications are developed as regular Android applications and we refactored to fit the OSGi paradigm.

6.1. Test application

A synthetic Test application is designed to investigate the possibility of saving energy by offloading an application task to a remote server, as well as to validate the power consumption model in the context of method offloading. The candidate method to offload generates artificial load on the CPU, performing some dummy calculations. Experiments are performed by changing argument and return value sizes, i.e. 10, 100 and 1000 KB. The application does not update any content on the screen to limit the influence of display unit (which is not relevant for taking the offloading decision).

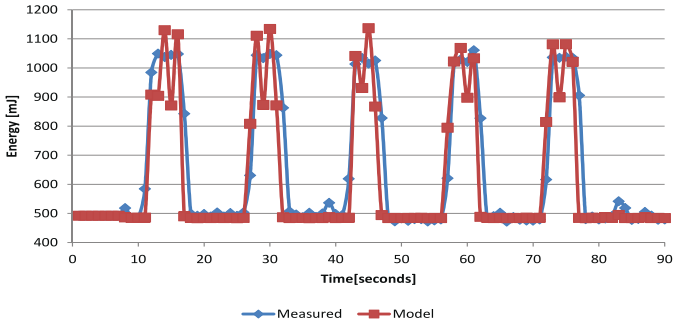
Fig. 10 represents the energy consumption for local and remote execution of each operation with 100 KB argument and return size. Each single peak in Fig. 10a amounts to the total energy required to perform each operation locally. Fig. 10b exhibits the energy consumption required to offload the operation to the remote server. Two small peaks of identical height appear for each offloaded operation. The identical height is due to similar size of argument and return value. The first peak occurs when operation is offloaded to the remote server and the second peak appears when processed data is returned from the remote server. The mobile device remains in idle state and waiting for the processed data during the peaks. It is observed that energy required to execute the operation locally is much higher than the remote execution.

Fig. 10 c shows the relative error per operation between measured and modeled energy consumption. The average error remains less than 10%.

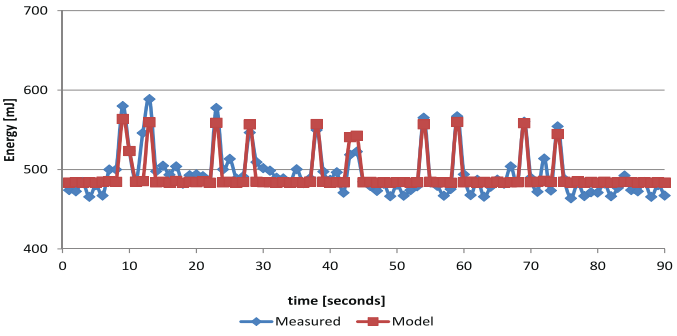
Fig. 11 represents energy gain achieved by offloading the operation. This energy gain is determined by comparing the energy consumed when an operation is executed locally and the remotely, and can be determined using following equation:

$$\text{Gain achieved [\%]} = \left| \frac{E_{Local} - E_{Offload}}{E_{Local}} \right| \tag{21}$$

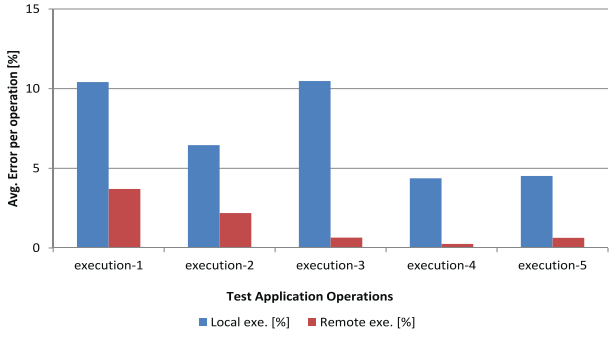
where E_{Local} and $E_{Offload}$ represents energy required for the local and the remote execution of the task, respectively.



(a) Energy consumption per execution when calculated locally



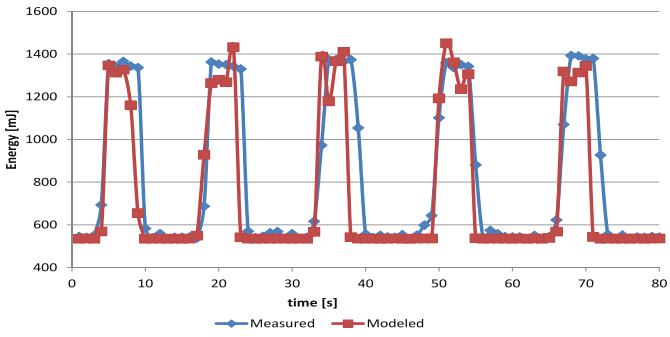
(b) Energy consumption per execution when offloaded to remote server



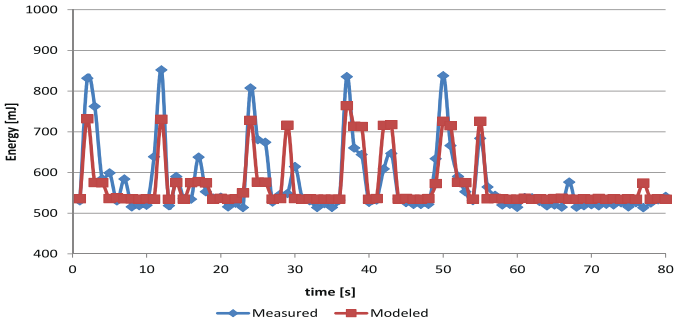
(c) Avg. Error per execution between measured and modeled power consumption

Fig. 10. Test application: energy consumption comparison between local method execution and offloading method to remote server with argument and return size value 100 KB (note scales on vertical axis).

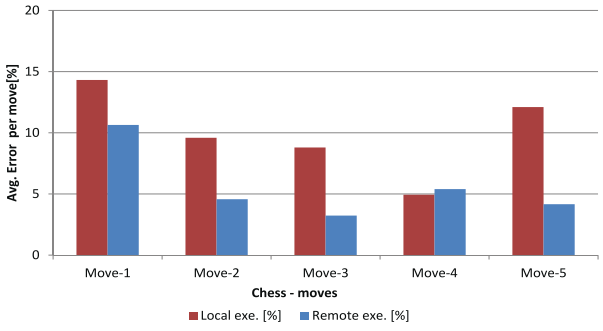
Fig. 11. Test application: average gain achieved for measured and modeled experimental results.



(a) Energy consumption per move when calculated locally



(b) Energy consumption per move when offloaded to remote server



(c) Average Error per move between measured and modeled power consumption

Fig. 12. Chess game: energy consumption comparison between local and offloaded moves to the remote server (note scales on vertical axis).

Results shows that 18–43% energy can be saved by offloading the operation depending upon argument and return size, server resources and the network conditions.

6.2. Chess game

The chess engine represents the core intelligence of the chess application, and we investigate whether it is possible to save energy by offloading the chess core algorithm (i.e. calculating the next move) to a remote server. Five consecutive chess moves were played to in order to determine the energy required to calculate each move locally as well as the energy needed to send a move to the server, and retrieve the calculated move from this server.

Fig. 12 represents the power consumption for local and remote calculation of each move. Each single peak in Fig. 12a exhibits total energy required to calculate move locally. Fig. 12b represents energy consumption required to offload the calculation to the remote server. Two small peaks appear for each offloaded calculation of the move. The first peak occurs when the user plays a move and data is

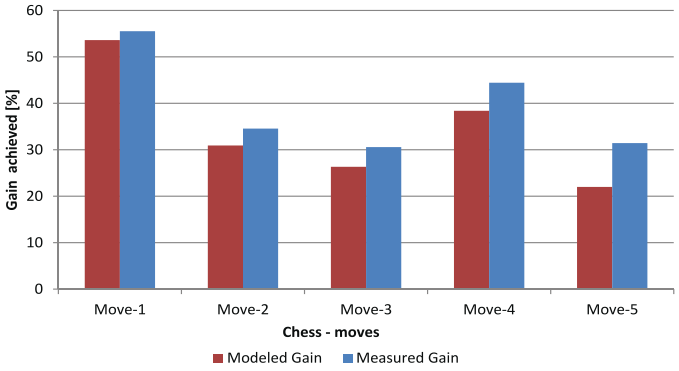


Fig. 13. Chess game: average gainachieved for measured and modeled experimental results.

transmitted over the network. The second peak occurs energy when processed data is returned from the remote server. The mobile device remains in the waiting state between these two peaks. It is clearly observed that energy required to calculate the move locally is much higher than for the remote calculation in spite of data exchange between the mobile device and the remote server, and mobile device in idle state while waiting for processed move from the server. It is also observed that the first peak is higher than the second one. This is explained by the fact that the first peak represents energy consumption required to play a move by the user, plus the transmission of data on the network, while the second peak only exhibits processed data returned from the remote server and the screen update.

Fig. 12 c represents the relative error per move between measured and modeled power consumption. The average error per move remains less than 10%.

The energy gain achieved by offloading is expressed in Fig. 13. This gain is calculated by comparing the energy consumption when move is calculated locally and with offloading to the remote server. Results shows that 25–55% energy can be saved by offloading move to the remote server depending on the server resources and the network conditions.

6.3. Photo Editor

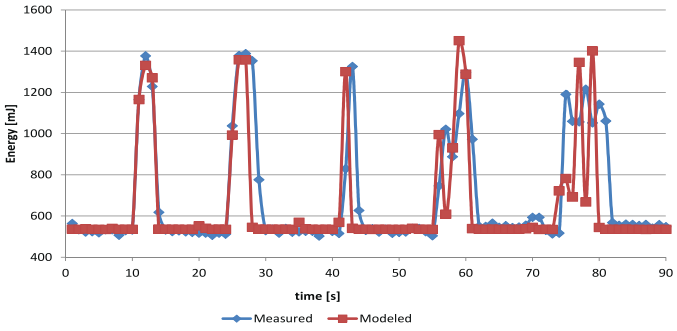
A Photo Editor is a popular application used on smartphone devices, which is mostly used to edit photographs before uploading to social networking sites. The Photo Editor application can perform several image filters such as blur, sharpening, RGB and histogram equalization on photos, and these operations require CPU processing.

To investigate whether the offloading is beneficial in terms of energy consumption for the Photo Editor application, these operations are performed locally on mobile device itself and remotely on a distant server. In the latter case, the full image is uploaded to the remote server for processing.

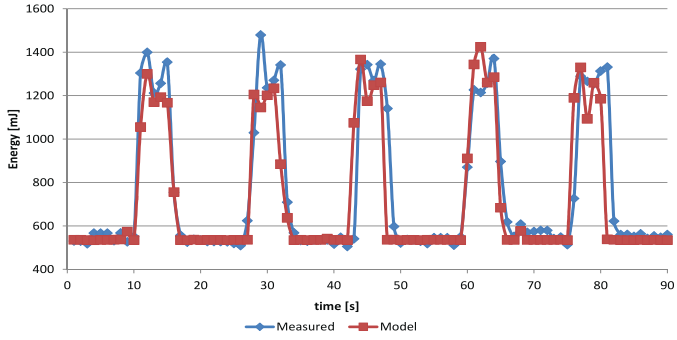
Fig. 14a and 14b presents results for local and the remote execution of the photo editor operations respectively. Each peak represents the execution of an operation (either locally or remotely depending on the graph). We observe an almost identical peak in power consumption, but notice that the peaks are broader in the remote execution case. This is due to additional time required for sending and receiving the entire image to and from the remote server. Therefore, it is concluded that more energy is consumed when the photo editor operations are performed remotely.

Fig. 14c represents relative error per operation between measured and modeled power consumption. Average error per operation remains less than 10%.

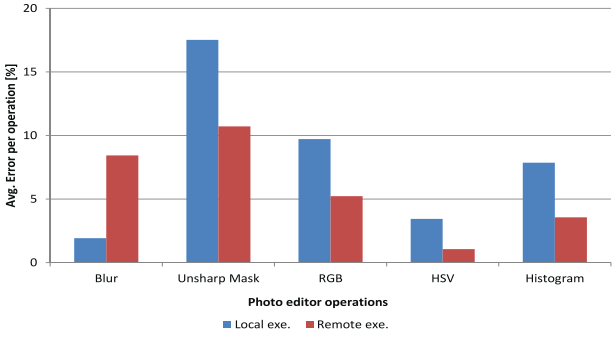
Fig. 15 presents total energy consumption per operation for both local and the remote execution. It is observed that 7–40% extra energy is consumed when executing the operation remotely. This is due



(a) Energy consumption per operation when calculated locally



(b) Energy consumption per operation when offloaded to remote server



(c) Avgerage Error per move between measured and modeled power consumption

Fig. 14. Photo Editor application: energy consumption comparison between local method execution and offloading method to remote server.

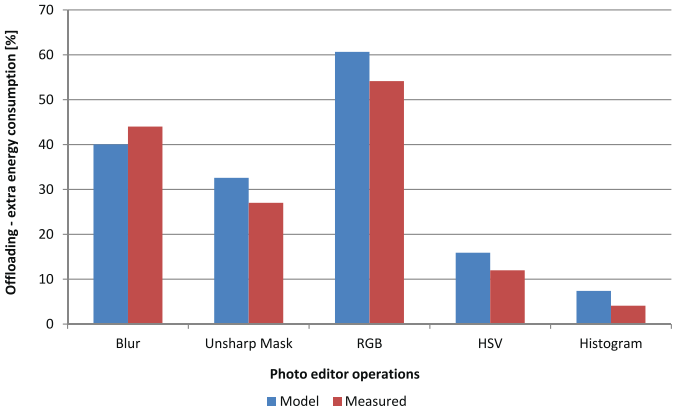


Fig. 15. Photo Editor: extra energy consumption for offloading to the remote server, comparison between measured and the modeled energy.

to heavy data transmission between mobile client and the remote server. We should however note that in these experiments, photos are uploaded in RAW format which contributes to heavy network traffic. Alternative solutions to save data transmission energy consist of sending and receiving photos in compressed format, as this would reduce energy required for bulky data transmission.

7. Conclusion and future work

In this paper, we have presented power models of mobile devices equipped with multi-core processors. These power models include the calibration of CPU, display unit, Wi-Fi and memory access [RAM] modules and can be used to estimate energy consumption of application running on the device itself.

The average error between the modeled and actually measured power consumption is typically smaller than 10%, in a wide variety of test conditions.

Application developers can use this power information to design energy efficient applications with the possibility of offloading computational intensive tasks to the remote server to save mobile device's energy resource. These power models are validated with different scenarios and applications. We have evaluated these power models by offloading application's task to the remote server using the AIOLOS offloading framework. It has been concluded that computational intensive applications can take benefit from offloading as compared to multimedia application where large data has to be transferred to the remote server for processing. Results shows that 18–55% of the mobile device energy consumption can be saved through offloading computational intensive tasks to the remote server depending upon the task, server resources and the current network conditions.

The power model currently captures the average energy consumed for each method call, and based on this average the offloading decision is made. To more accurately capture the stochastic execution of application, in principle a distribution of power consumption for each method call could be constructed. This would allow to make a more robust offloading decision, e.g. only considering offloading if the probability of energy consumption exceeds a certain threshold. In addition, AIOLOS (Verbelen et al., 2012) offers the possibility to take into account the size (in bytes) of method arguments and size of return value. Exploiting this also provides opportunities for the more fine-grained predictions and this work is considered as a future work. Furthermore, we plan to extend the wireless network energy models to cellular technologies (LTE, 4G) and validate these based on the application presented.

Acknowledgment

This project was partly funded by the UGent BOF-GOA project "Autonomic Networked Multimedia Systems".

References

Atluri, V., Cakmak, U., Lee, R., Varanasi, S., 2012.A new era of personalized computing.[Online] Available: https://tmt.mckinsey.com/content/publications. Accessed February, 2013.

Bornholt, J., Mytkowicz, T., McKinley, K.S., 2012. The model is not enough: understanding energy consumption in mobile devices. In: A Symposium on High Performance Chips (Hot Chips 2012). San Jose, CA, USA.

Chun, B.-G., Ihm, S., Maniatis, P., Naik, M., Patti, A., 2011. Clonecloud: elastic execution between mobile device and cloud. In: Proceedings of the Sixth International Conference on Computer systems. ACM, New York, NY, USA, pp. 301–314.

CPUfreq Governors. 2013. https://access.redhat.com/site/documentation/en-US/Red_Hat_Enterprise_Linux/6/html/Power_Management_Guide/cpufreq_governors.html. Accessed June, 2013.

Cuervo, E., Balasubramanian, A., Cho, D.-k., Wolman, A., Saroiu, S., Chandra, R., Bahl, P., 2010. Maui: making smartphones last longer with code offload. In: Proceedings of the 8th International Conference on Mobile Systems, Applications, and Services. ACM, New York, NY, USA, pp. 49–62.

Dou, A., Kalogeraki, V., Gunopulos, D., Mielikainen, T., Tuulos, V.H., 2010. Misco: a mapreduce framework for mobile systems. In: Proceedings of the 3rd International Conference on PErvasive Technologies Related to Assistive Environments. ACM, New York, NY, USA, pp. 32:1–32:8.

Fernando, N., Loke, S., Rahayu, W., 2013. Honeybee: a programming framework for mobile crowd computing. In: Zheng, K., Li, M., Jiang, H. (Eds.), Mobile and Ubiquitous Systems: Computing, Networking, and Services. In: Volume 120 of Lecture Notes of the Institute for Computer Sciences, Social Informatics and Telecommunications Engineering. Springer, Berlin, Heidelberg, pp. 224–236.

Gao, W., Li, Y., Lu, H., Wang, T., Liu, C., 2014. On exploiting dynamic execution patterns for workload offloading in mobile cloud applications. In: Proceedings of 2014 IEEE 22nd International Conference on Network Protocols (ICNP). IEEE, pp. 1–12.

Honza's Chess. 2013. http://honzovysachy.sourceforge.net. Accessed July, 2013.

Huang, J., Qian, F., Gerber, A., Mao, Z.M., Sen, S., Spatscheck, O., 2012. A close examination of performance and power characteristics of 4G LTE networks. In: Proceedings of the 10th International Conference on Mobile Systems, Applications, and Services. ACM, New York, NY, USA, pp. 225–238.

Jiao, L., Friedman, R., Fu, X., Secci, S., Smoreda, Z., Tschofenig, H., 2013. Cloud-based computation offloading for mobile devices: state of the art, challenges and opportunities. In: Proceedings of Future Network and Mobile Summit (FutureNetwork-Summit), 2013, pp. 1–11.

Jung, W., Kang, C., Yoon, C., Kim, D., Cha, H., 2012. Devscope: a nonintrusive and online power analysis tool for smartphone hardware components. In: Proceedings of the Eighth IEEE/ACM/IFIP International Conference on Hardware/Software Codesign and System Synthesis. ACM, New York, NY, USA, pp. 353–362.

Kantardzic, M., 2003. Chapter 10: Genetic algorithms. In: Fagerberg, J., Mowery, D., Nelson, R. (Eds.), In: Data Mining: Concepts, Models, Methods, and Algorithms. John Wiley Sons., Oxford.

Kemp, R., Palmer, N., Kielmann, T., Bal, H., 2012. Cuckoo: A computation offloading framework for smartphones. In: Mobile Computing, Applications, and Services. Springer, pp. 59–79.

Kim, M., Chung, S.W., 2013. Accurate GPU power estimation for mobile device power profiling. In: Proceedings of 2013 IEEE International Conference on Consumer Electronics (ICCE), pp. 183–184.

Kim, M., Kong, J., Chung, S.W., 2012. Enhancing online power estimation accuracy for smartphones. IEEE Trans. Consum. Electron. 58 (2), 333–339.

Kjrgaard, M., Blunck, H., 2012. Unsupervised power profiling for mobile devices. In: Puiatti, A., Gu, T. (Eds.), Mobile and Ubiquitous Systems: Computing, Networking, and Services. In: Volume 104 of Lecture Notes of the Institute for Computer Sciences, Social Informatics and Telecommunications Engineering. Springer, Berlin, Heidelberg, pp. 138–149.

Kosta, S., Aucinas, A., Hui, P., Mortier, R., Zhang, X., 2012. Thinkair: dynamic resource allocation and parallel execution in the cloud for mobile code offloading. In: Proceedings of IEEE International Conference on INFOCOM, 2012. IEEE, pp. 945–953.

Kumar, K., Lu, Y.-H., 2010. Cloud computing for mobile users: can offloading computation save energy? Computer 43 (4), 51–56.

Ma, X., Cui, Y., Wang, L., Stojmenovic, I., 2012. Energy optimizations for mobile terminals via computation offloading. In: Proceedings of 2012 2nd IEEE International Conference on Parallel Distributed and Grid Computing (PDGC), pp. 236–241.

Monsoon. 2013. http://www.msoon.com/LabEquipment/PowerMonitor/. Accessed March 2013.

Ou, Z., Dong, S., Dong, J., Nurminen, J.K., Ylä-Jääski, A., Wang, R., 2013. Characterize energy impact of concurrent network-intensive applications on mobile platforms. In: Proceedings of the Eighth ACM International Workshop on Mobility in the Evolving Internet Architecture. ACM, New York, NY, USA, pp. 23–28.

Ou, Z., Dong, J., Dong, S., Wu, J., Yla-Jaaski, A., Hui, P., Wang, R., Min, A., 2015. Utilize signal traces from others? a crowdsourcing perspective of energy saving in cellular data communication. IEEE Trans. Mob. Comput. 14 (1), 194–207.

Pathak, A., Hu, Y.C., Zhang, M., Bahl, P., Wang, Y.-M., 2011. Fine-grained power modeling for smartphones using system call tracing. In: Proceedings of the Sixth Conference on Computer Systems. ACM, New York, NY, USA, pp. 153–168.

Rellermeyer, J.S., Alonso, G., Roscoe, T., 2007. R-OSGI: distributed applications through software modularization. In: Proceedings of the ACM/IFIP/USENIX 2007 International Conference on Middleware. Springer-Verlag, New York, Inc., New York, NY, USA, pp. 1–20.

Shi, C., Lakafosis, V., Ammar, M.H., Zegura, E.W., 2012. Serendipity: Enabling remote computing among intermittently connected mobile devices. In: Proceedings of the Thirteenth ACM International Symposium on Mobile Ad Hoc Networking and Computing. ACM, New York, NY, USA, pp. 145–154.

Shi, C., Habak, K., Pandurangan, P., Ammar, M., Naik, M., Zegura, E., 2014. Cosmos: computation offloading as a service for mobile devices. In: Proceedings of the 15th ACM International Symposium on Mobile Ad Hoc Networking and Computing. ACM, New York, NY, USA, pp. 287–296.

Shye, A., Scholbrock, B., Memik, G., 2009. Into the wild: studying real user activity patterns to guide power optimizations for mobile architectures. In: Proceedings of the 42nd Annual IEEE/ACM International Symposium on Microarchitecture. ACM, New York, NY, USA, pp. 168–178.

Sklavos, N., Touliou, K., 2007. A system-level analysis of power consumption and optimizations in 3g mobile devices. In: Labiod, H., Badra, M. (Eds.), New Technologies, Mobility and Security. Springer, Netherlands, pp. 217–227.

The OSGi Alliance, 2009. OSGi Service Platform market, Core Specifications, Release 4, Version 4.2. The OSGi Alliance.

Vallina-Rodriguez, N., Crowcroft, J., 2013. Energy management techniques in modern mobile handsets. IEEE Commun. Surv. Tutor. 15 (1), 179–198.

Verbelen, T., Simoens, P., Turck, F.D., Dhoedt, B., 2012. AIOLOS: middleware for improving mobile application performance through cyber foraging. J. Syst. Softw. 85 (11), 2629–2639.

Yoon, C., Kim, D., Jung, W., Kang, C., Cha, H., 2012. Appscope: application energy metering framework for android smartphones using kernel activity monitoring. In: Proceedings of the 2012 USENIX Conference on Annual Technical Conference. USENIX Association, Berkeley, CA, USA, p. 36.

Zhang, L., Tiwana, B., Dick, R., Qian, Z., Mao, Z., Wang, Z., Yang, L., 2010. Accurate online power estimation and automatic battery behavior based power model generation for smartphones. In: Proceedings of 2010 IEEE/ACM/IFIP International Conference on Hardware/Software Codesign and System Synthesis (CODES+ISSS), pp. 105–114.

Farhan Azmat Ali got his M.Sc. degree from the Ostwestfalen-Lippe University of Applied Sciences, Germany and is continuing Ph.D. studies and research at the Department of Information Technology (INTEC) of the Faculty of Engineering at Ghent University, Ghent, Belgium since January, 2009. His main area of interest is mobile cloud computing, characterization of mobile sub-systems in terms of energy consumption and reliable video transmission over wireless networks.

Pieter Simoens is an assistant professor affiliated with the Department of Information Technology of the Ghent University and with iMinds. He is teaching courses on Mobile Application Development and Software Engineering. His main research interests include mobile cloud offloading, service-oriented networking, edge/fog computing paradigms, cloud robotics and service engineering for advanced mobile applications. In these fields, he is author and co-author of more than 70 papers published in international journals or in the proceedings of international conferences. He has also been involved in several national and European research projects.

Tim Verbelen received his M.Sc. degree in Computer Science from Ghent University, Belgium in June 2009. In July 2013, he received his Ph.D. degree with his dissertation "Adaptive Offloading and Configuration of Resource Intensive Mobile Applications". Since August 2009, he has been working at the Department of Information Technology (INTEC) of the Faculty of Engineering at Ghent University, and is now active as postdoctoral researcher at iMinds. His main research interests include mobile cloud computing, adaptive software and Internet of Things. As member of the iMinds Strategic IoT program, he works on enabling technologies for supporting the next generation IoT applications.

Piet Demeester is professor in the faculty of Engineering at Ghent University. He is head of the research group "Internet Based Communication Networks and Services" (IBCN). He is also leading the Future Internet (Networks, Media and Service) Department of the Interdisciplinary Institute for Broadband Technology (iMinds). After finishing a Ph.D. in 1988, he established a research group in this area working on different material systems (AlGaAs, InGaAsP, GaN). In 1992 he started research on communication networks and established the IBCN research group. The group is focusing on several advanced research topics: Network Modeling, Design & Evaluation; Mobile & Wireless Networking; High Performance Multimedia Processing; Autonomic Computing & Networking; Service Engineering; Content & Search Management and Data Analysis & Machine Learning. The research of IBCN resulted in about 50 Ph.D.s, 1250 publications in international journals and conference proceedings, 30 international awards and 4 spin-off companies.

Bart Dhoedt received his Master's degree in Electrotechnical Engineering (1990) from Ghent University. His research, addressing the use of micro-optics to realize parallel free space optical interconnects, resulted in a Ph.D. degree in 1995. After a 2-year post-doc in opto-electronics, he became Professor at the Department of Information Technology. He is responsible for various courses on algorithms, advanced programming, software development and distributed systems. His research interests include software engineering, distributed systems, mobile and ubiquitous computing, smart clients, middleware, cloud computing and autonomic systems. He is author or co-author of more than 300 publications in international journals or conference proceedings.